

We call ourselves **Homo sapiens** because our intelligence is so important to us.

The ultimate achievement in AI would be to construct a machine that can mimic or exceed human mental capabilities, including: Reasoning, understanding, imagination, recognition, creativity and emotions.

Human VS Rationally

Human thinking (Cognitive modelling) make the machine include human factors like emotions; **more to do with hypothesis & experiments, example: Neural Networks.**

Rationally thinking (Laws Of Thought) make the machine concerned with an ideal standard of correctness it uses formal logic; **more to do with mathematics & engineering.**

Turing test: A computer passes the test if a human interrogator, after posing some written questions, cannot tell whether the written responses come from a person or from a computer.

**An agent** is just something that acts.

**A rational agent** is one that acts so as to achieve the best outcome.

**Intelligence** is concerned mainly with rational action.

Percept: Refers to the content an agent's sensors are perceiving.

Percept sequence: The complete history of everything the agent has ever perceived.

Agent function: Describe the agent behavior.

**Task environments** are problems to which rational agents are the solutions.

To specify a task environment we define PEAS:

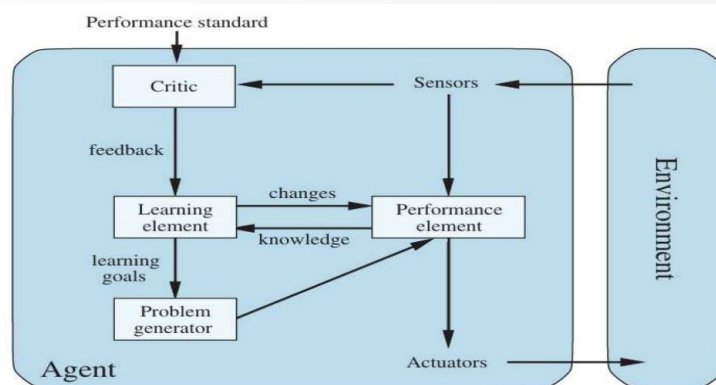
- Performance measure: How agent is assessed.
- Environment: What elements exist around agent.
- Actuators: How agent change the environment.
- Sensors: How agent sense the environment.

- **Fully Observable:** The agent's sensors have access to the complete state of the environment at each point in time. *Example: A chess game — the board and all pieces are visible.*
- **Partially Observable:** The agent has limited access to the full state due to noisy or limited sensors. *Example: A robot navigating through a dark room using sonar.*
- **Deterministic:** The next state of the environment is completely determined by the current state and the agent's action. *Example: Solving a mathematical equation.*
- **Stochastic:** The outcome may involve randomness or uncertainty. *Example: Weather prediction or financial markets.*
- **Episodic:** The agent's experience is divided into atomic episodes. Each episode consists of a percept and an action, and is independent of others. *Example: Image classification where each image is processed independently.*
- **Sequential:** The current decision affects future decisions. Tasks are interdependent. *Example: Chess or autonomous driving.*
- **Static:** The environment does not change while the agent is deliberating. *Example: Crossword puzzle.*
- **Dynamic:** The environment can change during decision-making. *Example: A robotic vacuum cleaning while people walk around.*
- **Discrete:** The environment has a finite number of distinct states, percepts, and actions. *Example: Board games.*
- **Continuous:** The environment can be described with real-valued variables. *Example: Robotic arm movement, self-driving vehicles.*
- **Single Agent:** The agent is the only entity acting in the environment. *Example: Solving a maze.*
- **Multi-Agent:** Multiple agents exist, possibly with competing or cooperating goals. *Example: Multiplayer video games or stock trading.*

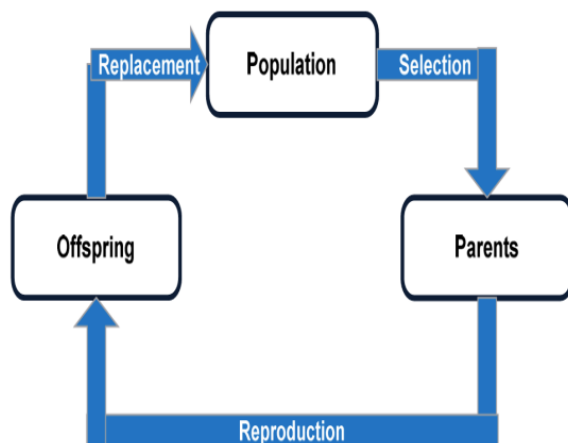
Agent basic types:

- Simple reflex agents: The simplest kind of agent; choose action based on the current percept, operate effectively in fully observable environments.
- Model-based reflex agents are agents that maintain an internal memory or state which helps them make decisions in situations where they cannot see or sense the entire environment.
- Goal-based agents: They act with a goal in mind.
- Utility-based agents: They use a utility function evaluate how desirable different outcomes are.
- Learning agents: They are intelligent systems that improve their performance over time through experience.

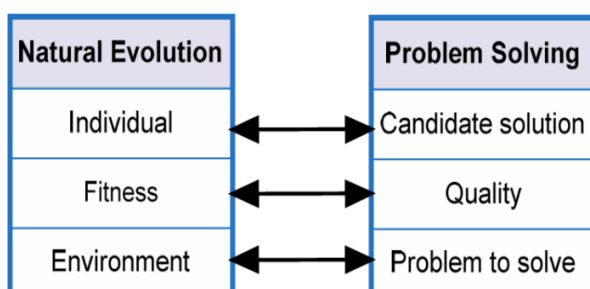
Learning agents figure



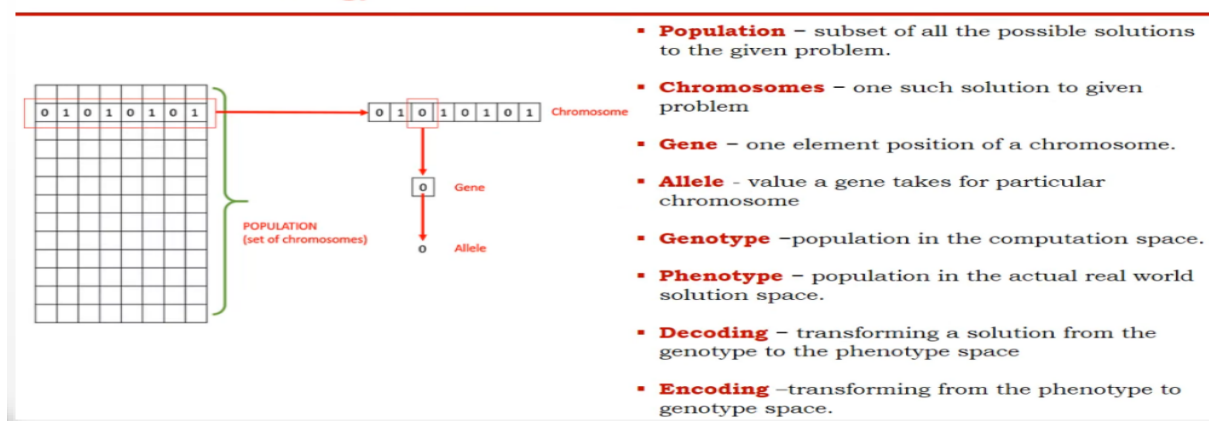
Evolutionary schema



The correspondence between natural evolution and problem solving



## Basic Terminology of GA



## Swarm

- Best fitness of each particle (pbest): the best solution (fitness) achieved so far by particle  $i$
- Best fitness of swarm (gbest): the best solution achieved so far by any particle in the swarm
- Velocity and position update: for exploring and exploiting the search space to locate the optimal solution.

- Position of particle ( $i$ ) is adjusted as

$$x_i^{(t+1)} = x_i^{(t)} + v_i^{(t+1)} \quad (1)$$

- Velocity of particle ( $i$ ) is updated as follows:

$$v_i^{(t+1)} = wv_i^{(t)} + c_1r_1(p_{(i,lb)}^{(t)} - x_i^{(t)}) + c_2r_2(p_{gb}^{(t)} - x_i^{(t)}) \quad (2)$$

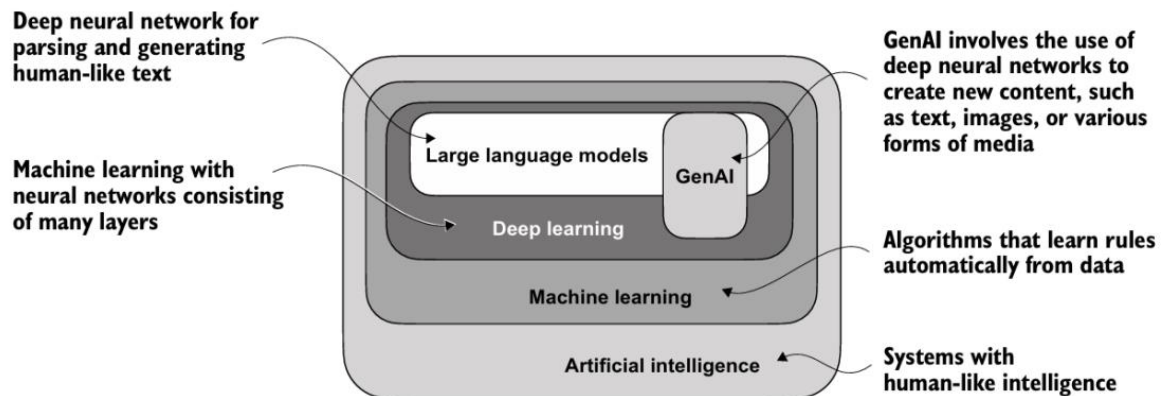
- $i$  is the  $i$ -th particle.
- $t$  is the generation counter.
- $v_i^{(0)}$  set randomly.
- $w$  adds to the inertia of the particle.
- $c_1$  and  $c_2$  are the acceleration coefficients.
- $r_1$  and  $r_2$  are random numbers  $\in [0, 1]$ .
- $p_{(i,lb)}^{(t)}$  is the local best of  $i$ -th particle.
- $p_{gb}^{(t)}$  is the global best.

$$v_i^{(t+1)} = wv_i^{(t)} + c_1r_1(p_{(i,lb)}^{(t)} - x_i^{(t)}) + c_2r_2(p_{gb}^{(t)} - x_i^{(t)})$$

- **Momentum part**,  $wv_i^{(t)}$ 
  - ▶ inertia component
  - ▶ memory of previous flight direction
  - ▶ prevents particle from drastically changing direction
- **Cognitive part**,  $c_1r_1(p_{(i,lb)}^{(t)} - x_i^{(t)})$ 
  - ▶ quantifies performance relative to past performances
  - ▶ memory of previous best position
  - ▶ nostalgia
- **Social part**,  $c_2r_2(p_{gb}^{(t)} - x_i^{(t)})$ 
  - ▶ quantifies performance relative to neighbors
  - ▶ envy

The order of the update process: Initialization > Evaluate > Update pBest > Update gBest > Update Velocity > Update Position.

Generative Ai (GenAi): Ai that can generate new content such as text, images, video, audio, or software code in response to a user's prompt.



How generative ai works:

- Training: To create a foundation model that can serve as the basis of multiple gen AI applications.
- Tuning: To tailor the foundation model to a specific gen ai tasks.
  - Fine tuning: Involves feeding the model labelled data (data with its correct answers) specific to the content generation application is likely to receive.
  - Reinforcement learning with human feedback: Human users respond to generated content with evaluations the model can use itself for greater accuracy.
- Generative, evaluation, and returning: To asses the gen Ai output and continually improve its quality and accuracy.

Discriminative VS Generative model

- Discriminative models: Are used in supervised learning in which the labels or categories of data are known
- Generative models: Are typically trained with unsupervised learning when they are fed a mass of unlabelled data and sort it by themselves.

Large language model (LLM): Are category of deep learning models trained on immense amounts of data . LLMs are built on a type of neural network architecture called **transformer**.

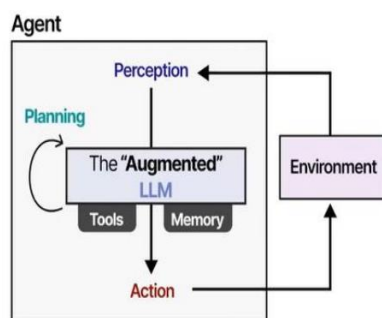
Types of transformer model:

- Encoder-Only Models: Use just the encoder part of the transformer architecture. Designed to understand the input but they don't generate new text. Great for text classification or extracting information, and Extractive Question Answering. Example model: BERT.
- Decoder-Only Models: Use just the decoder part of the transformer architecture. Designed for generating text or predicting the next word in a sequence. Great for tasks like writing stories or auto-completing sentences. Example model: GPT, GPT-2, GPT-3.
- Encoder-Decoder Models: These models use both the encoder and decoder parts of the transformer. Great for tasks like translating a sentence into another language or summarizing a text. Example model: T5(Text-To-Text Transfer Transformer).

Architecture based LLMs:

- Autoregressive: such as the GPT (Generative Pre-trained Transformer) series, generate text by predicting the next token in a sequence based on the preceding tokens. They use a **probability distribution** to select the most likely next word or character
- Autoencoding: such as BERT (Bidirectional Encoder Representations from Transformers), grasp the context of words in a sentence by predicting **masked** tokens.
- Seq2Seq: Like T5 (Text-To-Text Transfer Transformer), are designed for tasks where the input and output are sequences, such as **translation, summarization, and text generation**. They typically consist of an **encoder and a decoder**. The encoder processes the input sequence, and the decoder generates the output sequence.

Agent: In the context of GenAI we define agents to be GenAI systems that serve a user's goals via actions that engage with systems outside the GenAI itself. This GenAI is usually a LLM.



Anatomy of prompt (RICIC):

- Role: Persona.
- Instruction: What to do ?
- Context: Additional information describing the instructions
- Input data: Content to be processed.
- Constraints: Format, style, length

Templates:  $f(x)$  that create prompts by replacing variables

Instruction-Based Prompting: Prompting is often used to have the LLM answer a specific question or resolve a certain task.

Specificity: Accurately describe what you want to achieve.

Hallucination: LLMs may generate incorrect information confidently.

Order: Either begin or end your prompt with the instruction. Especially with long prompts, information in the middle is often forgotten. LLMs tend to focus on information either at the beginning of a prompt (primacy effect) or the end of a prompt (recency effect).

Chain Prompting: Breaking up the problem we can do so between prompts. Essentially, we take the output of one prompt and use it as input for the next, thereby creating a continuous chain of interactions that solves our problem.

Reasoning: Think before you answer.

## Tree-of-Thought: Exploring Intermediate Steps

ToT is a reasoning framework where the LLM generates multiple possible "next steps" at each stage, evaluates them, prunes bad branches, and explores promising ones — like exploring a tree of thoughts.

### The Four Core Components of ToT

| Component             | Function                          | Example                                                      |
|-----------------------|-----------------------------------|--------------------------------------------------------------|
| Thought Decomposition | Break problem into smaller steps  | "Step 1: Choose which jug to pour from"                      |
| Thought Generator     | Generate N possible next thoughts | "Option A: Pour 8→3, Option B: Pour 8→5, Option C: Pour 3→5" |
| State Evaluator       | Score each thought (0-1)          | "A: 0.3 (wasteful), B: 0.7 (promising), C: 0.2 (invalid)"    |
| Search Algorithm      | Decide which branches to explore  | BFS, DFS, or beam search (keep top K)                        |

LangChain: Is an open-source framework for building applications based on large language models (LLMs).

Table 7-1. The pros and cons of different memory types.

| Memory type                  | Pros                                                                                                                                                                                     | Cons                                                                                                                                                                                                                    |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Conversation Buffer          | <ul style="list-style-type: none"><li>• Easiest implementation</li><li>• Ensures no information loss within context window</li></ul>                                                     | <ul style="list-style-type: none"><li>• Slower generation speed as more tokens are needed</li><li>• Only suitable for large-context LLMs</li><li>• Larger chat histories make information retrieval difficult</li></ul> |
| Windowed Conversation Buffer | <ul style="list-style-type: none"><li>• Large-context LLMs are not needed unless chat history is large</li><li>• No information loss over the last <math>k</math> interactions</li></ul> | <ul style="list-style-type: none"><li>• Only captures the last <math>k</math> interactions</li><li>• No compression of the last <math>k</math> interactions</li></ul>                                                   |
| Conversation Summary         | <ul style="list-style-type: none"><li>• Captures the full history</li><li>• Enables long conversations</li><li>• Reduces tokens needed to capture full history</li></ul>                 | <ul style="list-style-type: none"><li>• An additional call is necessary for each interaction</li><li>• Quality is reliant on the LLM's summarization capabilities</li></ul>                                             |

### The Driving Power Behind Agents: Step-by-step Reasoning

- ❑ **ReAct is a powerful framework that combines two important concepts in behavior: reasoning and acting.**
- ❑ The framework consists of iteratively following these three steps:
  - **Thought**
  - **Action**
  - **Observation**

A React system starts with a user prompt. The LLM processes the prompt and decides whether it can answer directly or if it needs more information. If it needs more information, it identifies a tool — like a calculator or a search engine or an API — that can help. The system then calls the tool, retrieves the necessary data, and reasons through the results to generate a final response. This back-and-forth between reasoning and action is what makes React powerful.

Semantic search: Enables searching by meaning and not simply keyword matching.

One of the leading ways to reduce **hallucinations** is to build a systems that can retrieve relevant information and provide to the LLM. This method called **RAG**.

Dense retrieval strategy for **RAG systems**: Convert an external knowledge base into a vector database by chunking a document and then embedding each chunk.

- Chunking long text: Use a single vector to represent the whole document.
- Multiple vectors per document (That is a better approach): We chunk the document into smaller pieces, and embed those chunks.
- Nearest neighbour search: Once the query is embedded, we need to find the nearest vectors to it from our text archive